

Day 3 Integrated Task

Student Exam Result Analytics System

Question

You are building a backend result processing module for a training institute.

You are given a collection of Student objects with the following fields:

studentId

studentName

course

marks

passed // true or false

Create 12 students with mixed courses, marks, and pass/fail status.

Perform the Following Operations

1. Store all students in a List implementation
2. Extract unique courses using a Set
3. Group students by course using a Map
4. Filter all passed students using Streams + Lambda
5. From passed students calculate:

- Total marks
- Average marks
- Highest marks scorer

6. Sort passed students:

- By marks (Descending)
- Then by studentName (Ascending)

Use Comparator + method reference

7. Generate course-wise total marks report

Create Map<course, totalMarks>

8. Use Optional to find student with studentId = 110

9. Create a functional interface to filter

Students with marks ≥ 75

10. Use method reference to print final sorted result list

Sample Input Dataset (for expected output reference)

ID

Name

Course

Marks

Passed

101

Anu

Java

82

true

102

Bala

Java

45

false

103

Charan

Python

91

true

104

Divya

Java

67

true

105

Esha
Python
38
false

106

Farhan
DevOps
74
true

107

Gokul
DevOps
88
true

108

Hari
Java
53
true

109

Isha
Python
79
true

110

John

DevOps

62

true

111

Kavya

Java

95

true

112

Lokesh

Python

49

false

Expected Output

Unique Courses (Set)

[Java, Python, DevOps]

Students Grouped by Course (Map)

Java → [Anu, Bala, Divya, Hari, Kavya]

Python → [Charan, Esha, Isha, Lokesh]

DevOps → [Farhan, Gokul, John]

Passed Students (Filtered)

Anu, Charan, Divya, Farhan, Gokul, Hari, Isha, John, Kavya

Total Marks of Passed Students

701

Average Marks of Passed Students

77.8

Highest Marks Scorer

Kavya – 95

Sorted Passed Students (Marks ↓, Name ↑)

Kavya – 95

Charan – 91

Gokul – 88

Anu – 82

Isha – 79

Farhan – 74

Divya – 67

John – 62

Hari – 53

Course-wise Total Marks (Map)

Java → 342

Python → 208

DevOps → 224

Optional Search Result (studentId = 110)

Student found: John – DevOps – 62

Students with Marks ≥ 75 (Functional Interface Filter)

Anu – 82

Charan – 91

Gokul – 88

Isha – 79

Kavya – 95

Final Report Printed Using Method Reference

(Displays the sorted passed student list)

SOLUTION

```
package studentdemo;

import java.util.*;
import java.util.stream.Collectors;

public class StudentPerformance {
    public static void main(String[] args) {
        List<StudentReport> studentList = new ArrayList<StudentReport>();
        StudentReport Stu1 = new StudentReport(101, "Anu", "Java", 82, true);
        studentList.add(Stu1);
        StudentReport Stu2 = new StudentReport(102, "Bala", "Java", 45, false);
        studentList.add(Stu2);
        StudentReport Stu3 = new StudentReport(103, "Charan", "Python", 91, true);
        studentList.add(Stu3);
        StudentReport Stu4 = new StudentReport(104, "Divya", "Java", 67, true);
        studentList.add(Stu4);
        StudentReport Stu5 = new StudentReport(105, "Esha", "Python", 38, false);
        studentList.add(Stu5);
        StudentReport Stu6 = new StudentReport(106, "Farhan", "DevOps", 74, true);
        studentList.add(Stu6);
        StudentReport Stu7 = new StudentReport(107, "Gokul", "DevOps", 88, true);
        studentList.add(Stu7);

        StudentReport Stu8 = new StudentReport(108, "Hari", "Java", 53, true);
        studentList.add(Stu8);
        StudentReport Stu9 = new StudentReport(109, "Isha", "Python", 79, true);
        studentList.add(Stu9);
        StudentReport Stu10 = new StudentReport(110, "John", "DevOps", 62, true);
        studentList.add(Stu10);
        StudentReport Stu11 = new StudentReport(111, "Kavya", "Java", 95, true);
        studentList.add(Stu11);
        StudentReport Stu12 = new StudentReport(112, "Lokesh", "Python", 49, false);
        studentList.add(Stu12);
        //System.out.println(studentList);
        for (StudentReport s : studentList) {

            System.out.println("ID="+s.studentId+"\n\t"+"Name="+s.studentName+"\n\t"+"Course="+s.course+"\n\t"+"Mark="+s.mark+"\n\t"+"Passed="+s.result);
        }
    }
}
```

```

    }
    Set<String> stuCourse = new HashSet<>();
    for (StudentReport s : studentList) {
        stuCourse.add(s.getCourse());
    }
    System.out.println("Unique Courses are:");
    System.out.println(stuCourse);
    Map<String, List<StudentReport>> courseGroup =
        studentList.stream()
            .collect(Collectors.groupingBy(StudentReport::getCourse));

    System.out.println("\n" + "Students Grouped by Course (Map)");

    for (String course : courseGroup.keySet()) {

        List<String> names = courseGroup.get(course)
            .stream()
            .map(StudentReport::getStudentName)
            .collect(Collectors.toList());

        System.out.println(course + " → " + names);
    }

    List<StudentReport> passedStudent = studentList.stream()
        .filter(s -> s.mark > 50)
        .collect(Collectors.toList());

    System.out.println("\n\n" + "Passed Students (Filtered)");

    String names = passedStudent.stream()
        .map(StudentReport::getStudentName) // or s -> s.name
        .collect(Collectors.joining(", "));

    System.out.println(names);
    long noOfPassed = passedStudent.stream().count();
    System.out.println("\n" + "No.of Students Passes=" + noOfPassed);
    int TotalMarks = passedStudent.stream().map(s -> s.mark).reduce(0, Integer::sum);
    System.out.println("TotalMarks of Passed Students=" + TotalMarks);
    double averageMarks = passedStudent.stream().mapToInt(s1 ->
s1.mark).average().orElse(0);
    System.out.println("Average Marks=" + averageMarks);
    StudentReport HighestScorer =
passedStudent.stream().max(Comparator.comparingInt(s2 -> s2.mark)).orElse(null);
    System.out.println("Highest Scorer=" + HighestScorer.getStudentName()+"-
"+HighestScorer.getMark());

    List<StudentReport>
sortedPassed=passedStudent.stream().sorted(Comparator.comparing(StudentReport::getMark)
).toList().reversed();
    System.out.println("\n" + "Student Passed : Sorted By Mark- Descending");
    for(StudentReport s:sortedPassed) {
        System.out.println(s.getStudentName()+"-"+s.getMark());
    }
    List<StudentReport>
sortedByCourse=sortedPassed.stream().sorted(Comparator.comparing(StudentReport::getStu

```

```

dentName)).toList();
System.out.println("\n"+"Sorted by Name Ascending");
for(StudentReport s:sortedByCourse){
    System.out.println(s.getStudentName()+" "+s.getMark());
}
Map<String, Integer> courseWiseTotal =
    studentList.stream()
        .collect(Collectors.groupingBy(
            StudentReport::getCourse,
            Collectors.summingInt(StudentReport::getMark)));

System.out.println("\n"+"Course-wise Total Marks (Map)");

courseWiseTotal.forEach((course, total) ->
    System.out.println(course + " → " + total));
int targetId=101;
Optional<StudentReport> studentSearch =
    studentList.stream()
        .filter(s -> s.studentId == targetId)
        .findFirst();

System.out.println("\n"+"Optional Search Result (studentId = " + targetId + ")");

if (studentSearch.isPresent()) {
    StudentReport s = studentSearch.get();
    System.out.println("Student found: "
        + s.getStudentName() + " – "
        + s.getCourse() + " – "
        + s.getMark());
} else {
    System.out.println("Student not found");
}
StudentCheck filter1 = s -> s.mark >= 75;

System.out.println("Students with Marks ≥ 75 (Functional Interface Filter)");

studentList.stream()
    .filter(filter1::check)
    .forEach(s -> System.out.println(s.studentName + " – " + s.mark));

System.out.println("Final Report Printed Using Method Reference");
System.out.println("(Displays the sorted passed student list)");

List<StudentReport> passedStudent1 = studentList.stream()
    .filter(s -> s.mark > 50)
    .sorted(Comparator.comparing(StudentReport::getStudentName))
    .collect(Collectors.toList());

System.out.println("Final Report Printed Using Method Reference");
System.out.println("(Displays the sorted passed student list)");

passedStudent1.forEach(System.out::println);

```

```
}  
  
}
```

OUTPUT :

```
C:\Users\HP\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:D:\IntelliJ IDEA  
2026.1.1\lib\idea_rt.jar=59923" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -  
Dsun.stderr.encoding=UTF-8 -classpath
```

```
C:\Users\HP\IdeaProjects\HospitalMgmtSystems\out\production\HospitalMgmtSystems  
studentdemo.StudentPerformance
```

ID=101

Name=Anu

Course=Java

Mark=82

Passed=true

ID=102

Name=Bala

Course=Java

Mark=45

Passed=false

ID=103

Name=Charan

Course=Python

Mark=91

Passed=true

ID=104

Name=Divya

Course=Java

Mark=67

Passed=true

ID=105

Name=Esha

Course=Python

Mark=38

Passed=false

ID=106

Name=Farhan
Course=DevOps
Mark=74
Passed=true

ID=107

Name=Gokul
Course=DevOps
Mark=88
Passed=true

ID=108

Name=Hari
Course=Java
Mark=53
Passed=true

ID=109

Name=Isha
Course=Python
Mark=79
Passed=true

ID=110

Name=John
Course=DevOps
Mark=62
Passed=true

ID=111

Name=Kavya
Course=Java
Mark=95
Passed=true

ID=112

Name=Lokesh
Course=Python
Mark=49
Passed=false

Unique Courses are:

[Java, DevOps, Python]

Students Grouped by Course (Map)

Java → [Anu, Bala, Divya, Hari, Kavya]

DevOps → [Farhan, Gokul, John]

Python → [Charan, Esha, Isha, Lokesh]

Passed Students (Filtered)

Anu, Charan, Divya, Farhan, Gokul, Hari, Isha, John, Kavya

No.of Students Passes=9

TotalMarks of Passed Students=691

Average Marks=76.77777777777777

Highest Scorer=Kavya-95

Student Passed : Sorted By Mark- Descending

Kavya-95

Charan-91

Gokul-88

Anu-82

Isha-79

Farhan-74

Divya-67

John-62

Hari-53

Sorted by Name Ascending

Anu 82

Charan 91

Divya 67

Farhan 74

Gokul 88

Hari 53

Isha 79

John 62

Kavya 95

Course-wise Total Marks (Map)

Java → 342

DevOps → 224

Python → 257

Optional Search Result (studentId = 101)

Student found: Anu – Java – 82

Students with Marks ≥ 75 (Functional Interface Filter)

Anu – 82

Charan – 91

Gokul – 88

Isha – 79

Kavya – 95

Final Report Printed Using Method Reference

(Displays the sorted passed student list)

Final Report Printed Using Method Reference

(Displays the sorted passed student list)

Anu – Java – 82

Charan – Python – 91

Divya – Java – 67

Farhan – DevOps – 74

Gokul – DevOps – 88

Hari – Java – 53

Isha – Python – 79

John – DevOps – 62

Kavya – Java – 95

Process finished with exit code 0